Implementation Report

1. Short Implementation Summary

In this stage of the project, the mathematical formulation of the marketing campaign budget allocation problem has been successfully translated into a working Python implementation. The model was implemented using the Gurobi optimization solver through its Python API.

The current implementation includes:

- Definition of campaign level input data using a small synthetic test instance
- Construction of binary decision variables for campaign selection
- Implementation of the objective function to maximize risk adjusted return
- Integration of all model constraints, including budget, minimum reach, and channel capacity limits
- Execution of the solver and extraction of solution results

The implementation has been tested on a small instance consisting of 10 campaigns across 4 channels. The solver successfully returned an optimal solution, demonstrating that the model, data, and solver are correctly connected and functioning.

2. Model to Code Connection

The mathematical model developed in the previous stage has been directly mapped into code components. Each part of the formulation corresponds to a specific implementation element.

- **Decision Variables**
  In the mathematical model, $x_i \in \{0,1\}$represents whether campaign $i$is selected.
  In the implementation, this is defined as a binary variable for each campaign using Gurobi:

```
x = model.addVars(campaign_ids, vtype=GRB.BINARY)
```

- **Objective Function**
  The objective is to maximize risk-adjusted return:

$$\max \sum_i (RO\,I_i - \lambda \cdot Risk_i)x_i$$

This is implemented using:

```
gp.quicksum((roi[i] - risk_penalty * risk[i]) * x[i] for i in campaign_ids)
```

- **Budget Constraint**
  The total cost of selected campaigns must not exceed the available budget:

$$\sum_i Cos\,t_i x_i \leq B$$

Implemented as:

```
gp.quicksum(cost[i] * x[i] for i in campaign_ids) <= budget
```

- **Minimum Reach Constraint**
  The selected campaigns must achieve at least a minimum total reach:

$$\sum_i Reac\, h_i x_i \geq R_{min}$$

Implemented as:

```
gp.quicksum(reach[i] * x[i] for i in campaign_ids) >= min_reach
```

- **Channel Constraints**
  Each channel has an upper limit on the number of selected campaigns:

$$\sum_{i \in C_k} x_i \leq L_k$$

Implemented using a loop over channels:

- **Channel Constraints**
  Each channel has an upper limit on the number of selected campaigns:

$$\sum_{i \in C_k} x_i \leq L_k$$

Implemented using a loop over channels:

```
for ch, cap in channel_caps.items():
    gp.quicksum(x[i] for i in campaign_ids if channel[i] == ch) <= cap
```

This direct mapping ensures consistency between the mathematical formulation and its computational implementation.

3. Challenges Encountered

During the implementation process, several challenges were encountered:

- **Model Feasibility**
  Ensuring feasibility was a key concern. If the budget is too low or the minimum reach is too high, the model may become infeasible. Therefore, parameters were adjusted to create a balanced and solvable instance.

- **Coding Structure**
  Translating the mathematical formulation into a structured Python implementation required organizing data into dictionaries and ensuring that all indices (campaigns and channels) were correctly aligned across parameters and constraints.

- **Solver Integration**
  Proper integration of the Gurobi solver was necessary to ensure that the model could be built and solved correctly. This included handling solver status outputs and validating that the returned solution satisfies all constraints.

4. Implementation Results and Test Instance

The implemented model was tested on a small synthetic instance designed to validate the correctness of the model code solver integration.

The test instance consists of 10 marketing campaigns distributed across 4 different channels (Social, Search, Email, and TV). Each campaign is characterized by its cost, expected return (ROI), risk score, and expected reach. In addition, a total budget constraint, a minimum reach requirement, and channel-specific upper bounds on campaign selection were defined.

The model was solved using the Gurobi optimization solver through its Python interface. The solver successfully returned an optimal solution within a negligible runtime (less than one second), indicating that the implementation is computationally efficient for small instances.

The results obtained from the solver are as follows:

- **Solver Status:** Optimal
- **Objective Value:** 126.20
- **Total Cost:** 95 (equal to the budget limit)
- **Total Reach:** 715 (above the minimum required reach of 650)
- **Selected Campaigns:** C2, C5, C7, C8, C9, C10

The solution satisfies all model constraints. The total cost exactly meets the budget limit, showing that the model efficiently utilizes the available resources. The total reach exceeds the minimum requirement, ensuring that the campaign portfolio achieves the desired exposure. Additionally, the number of selected campaigns within each channel respects the predefined channel capacity constraints.

Overall, the result is consistent with expectations. The model successfully selects a portfolio of campaigns that balances return and risk while adhering to all operational constraints. This confirms that the mathematical formulation has been correctly translated into code and that the solver integration is functioning as intended.

5. Next Improvements

In the next stage of the project, several improvements will be implemented to enhance both the robustness and practical applicability of the model.

The current implementation will be extended to handle larger and more realistic datasets, and the synthetic test data will be replaced with CSV based input data.

Additionally, sensitivity analysis will be conducted on key parameters such as the budget and the risk penalty parameter ((\lambda)) to better understand their impact on the optimal solution.

Multiple scenarios will be evaluated to analyze how campaign selection changes under different constraints.

The presentation of results will also be improved by incorporating tables and visualizations.

Finally, potential model extensions, such as partial budget allocation and additional operational constraints, will be explored to make the model more applicable to real world decision making contexts.